

Highlights

A Physics-Informed Neural Network Forward Solver for the Rough-Volatility Calibration Inverse Problem: Dimension Scaling on the Markovian-Lifted PDE and a Fast Warm-Start Hybrid

Aviral Sharma, Kamal Preet Singh, Vipul Sharma

- Calibrating rough-volatility models is an inverse problem with no finite-dimensional forward PDE; we solve a Markovian-lifted surrogate PDE with a physics-informed neural network (PINN) forward operator.
- The forward solver’s error grows only sub-linearly in the lifted dimension (< 11 px bp to $n=32$), giving a controllable, convergent approximation to the true forward map that the naive grid cannot match.
- An explicit error decomposition separates forward-solver (network) error from forward-model (lift) error, isolating what limits inverse-problem accuracy and by how much.
- A parametric PINN forward operator, differentiable in the model parameters, warm-starts the calibration inverse problem at $\sim 9\times$ fewer expensive forward-model evaluations.
- We report where the AI forward operator’s generalisation degrades (rough, high-vol-of-vol parameter corners) so its use inside the inverse problem is bounded honestly, not oversold.

A Physics-Informed Neural Network Forward Solver for the Rough-Volatility Calibration Inverse Problem: Dimension Scaling on the Markovian-Lifted PDE and a Fast Warm-Start Hybrid

Aviral Sharma^{a,*}, Kamal Preet Singh^b, Vipul Sharma^c

^a*Chandigarh University, Chandigarh, India*

^b*SGT University, Gurugram, India*

^c*Independent Researcher, India*

Abstract

Calibrating a stochastic volatility model to an observed option surface is a classical inverse problem: given market prices, recover the parameters of a forward model that reproduces them. Rough-volatility models make this inverse problem unusually hard, because their forward problem has no finite-dimensional partial differential equation (PDE) to begin with—the variance is driven by a fractional, non-Markovian kernel. The standard remedy is a Markovian lift, which approximates the fractional kernel by n exponentials and recovers a genuine, but $(n+1)$ -dimensional, forward PDE. We build the forward operator for the resulting calibration inverse problem from a physics-informed neural network (PINN): a tanh-activated multilayer perceptron trained to satisfy the lifted PDE’s residual at randomly sampled collocation points, avoiding any computational grid. Because an AI forward operator is only as trustworthy as its own approximation error, we first characterise that error on its own terms, asking whether it degrades with the lifted dimension the way a finite-difference grid would. It does not: measured against an independent lifted Monte Carlo at the same n , and cleanly separated from the model-approximation error of the lift itself, the network’s solve-error grows only sub-linearly—from 5.4 ± 1.0 to 10.7 ± 2.0 price basis points as n climbs from 4 to 32—while a naive finite-difference grid needs

*Corresponding author

Email address: aviral98765@gmail.com (Aviral Sharma)

50^{n+1} points and is infeasible by $n=8$. We detail the four architecture and training choices needed for this convergence behaviour to hold in practice: a hard-constrained initial-condition ansatz, an integrated-variance baseline anchor, self-adaptive loss weighting, and a causal time-to-maturity curriculum. We then embed a parametric extension of this forward operator—one that also takes the five rough-Heston parameters as inputs and is differentiable in them—inside the calibration inverse problem itself. Used alone it is fast but insufficiently accurate to calibrate independently; used as a differentiable warm start for a short local refinement against the true (Fourier) forward model, it recovers full calibration quality at approximately nine times fewer expensive forward-model evaluations. All experiments calibrate to live Bitcoin option data from the Deribit exchange, recovering a genuinely rough Hurst exponent ($H \approx 0.09$) with strong negative correlation ($\rho \approx -0.79$).

Keywords: inverse problems, physics-informed neural networks, rough volatility, Markovian lift, fractional kernels, model calibration, curse of dimensionality, scientific machine learning

1. Introduction

Model calibration is the canonical inverse problem of mathematical finance: given a finite set of noisy, arbitrage-constrained option prices, recover the parameters of a forward model that reproduces them. Rough-volatility models make this inverse problem unusually severe, because their *forward* problem is already pathological. Motivated by the empirical roughness of realised variance [1, 2] and the steep short-maturity skew that classical stochastic-volatility models such as Heston’s [3] cannot reproduce, rough-volatility models drive the variance with a fractional noise whose Hurst exponent $H < 1/2$. That single fact—whose lineage runs back to long-memory volatility [4]—makes the variance process neither Markovian nor a semimartingale, so there is no finite-dimensional forward PDE to solve, let alone invert [1, 5].

The standard engineering escape is the *Markovian lift*: approximate the fractional kernel by a sum of n exponentials and recover a genuine $(n+1)$ -dimensional forward PDE [6, 7]. This turns an ill-posed forward problem into a well-posed but high-dimensional one, and pushes the difficulty of the inverse problem onto the cost of repeatedly solving that high-dimensional forward PDE inside a calibration loop—once per candidate parameter vec-

tor, at every iteration of the outer optimiser. Any AI method proposed as a forward operator for this inverse problem must therefore be judged on two separate questions: how accurately does it solve the forward PDE it is handed, and how does that accuracy behave as the lifted dimension n —the knob that trades model fidelity against forward-problem cost—is turned up. Both questions are usually left unanswered when a neural network is dropped into a calibration pipeline as a black-box pricer.

We answer them directly for a physics-informed neural network (PINN) forward operator. Rather than laying down a grid, a PINN represents the solution as a neural network and trains it to satisfy the PDE at randomly sampled collocation points [8], sidestepping the exponential blow-up that kills finite differences in high dimensions, in the spirit of the deep Galerkin and deep BSDE solvers [9, 10]. Because it is differentiable in its inputs, the same trained network doubles as a differentiable forward operator for gradient-based calibration once the parameters are added as additional inputs. Our concrete contributions are as follows.

1. **A convergence and stability study of the AI forward operator.** We show that the PINN’s own solve-error—the part of the total error under the network’s control—grows only sub-linearly as the lifted dimension climbs, staying below roughly eleven price basis points out to $n=32$, while a naive full-tensor finite-difference forward solver (50^{n+1} points) is already infeasible by $n=8$. We identify the four training design choices without which this convergence behaviour does not hold (Section 4).
2. **An error decomposition that isolates what limits the inverse problem.** We split total pricing error into the network’s own solve-error (measured against a lifted Monte Carlo at the same n) and the forward model’s lift-error (measured against the exact, non-lifted Fourier forward map). Their crossover gives a concrete, maturity-dependent rule for how large n —and hence how expensive the forward operator—needs to be before the network, not the model, is the accuracy bottleneck.
3. **A differentiable forward operator embedded in the calibration inverse problem.** A parametric extension of the network, additionally conditioned on the five rough-Heston parameters, is used both as a fast stand-alone forward map and as a warm start for a short local refinement against the true forward model, and we report where its

accuracy generalises and where it does not.

4. **A real-market solution of the inverse problem.** Calibrated to live Deribit Bitcoin option data, the recovered parameters return a genuinely rough Hurst exponent and a strongly negative skew, and we report the fit quality honestly rather than only the speed-up.
5. **Fast recalibration, quantified against the true forward model.** The parametric warm start plus a short exact-model polish recovers from-scratch calibration quality at approximately nine times fewer expensive forward-model evaluations; we state plainly where the standalone AI forward operator is not accurate enough to trust alone.

We are explicit about what is *not* new. Papantoleon and Rou [11] solved the lifted rough-Heston forward PDE with a mesh-free network before us. Our contribution is a residual PINN forward operator characterised specifically as a component of the calibration inverse problem: a quantitative convergence-versus-dimension study, an error decomposition that separates network error from model error, a real-market inverse-problem solution, and the warm-start hybrid.

2. Related work

Closest prior work.. Papantoleon and Rou [11] price European options in rough-diffusion models by recasting the forward pricing PDE as an energy-minimisation problem and approximating it in a time-stepping deep gradient flow, demonstrated on the lifted Heston model. Their method is the closest to ours in target and spirit; it differs in mechanism (time-stepping variational scheme versus residual PINN) and does not study forward-operator dimension scaling, error decomposition, or the calibration inverse problem downstream.

Neural forward operators for rough volatility.. Jacquier and Žurič [12] cast the path-dependent pricing equations as backward SDEs and solve them with a random-feature network, reducing training to a least-squares regression. They share the forward target but not the mechanism, and neither dimension scaling nor the inverse problem is studied.

AI methods for the calibration inverse problem.. The dominant approach for fast rough-volatility calibration is the supervised surrogate of Bayer et al. [13] and Horvath et al. [14]: a network trained offline on precomputed prices or

implied volatilities, used as a fast forward map inside the calibration loop. It is purely data-driven; the network never sees the governing equation, carries no PDE consistency, and—because it is not derived from a physics-informed forward solve—cannot be decomposed into a network-error and a model-error term the way ours can, which is precisely the diagnostic an inverse-problem user needs to trust or distrust a calibrated fit.

PINN training pathologies and remedies.. PINNs are well-known to fail as forward operators when loss terms sit on different scales or temporal causality is violated [15, 16]. The two stabilisers we use to keep the forward operator convergent and stable—self-adaptive homoscedastic-uncertainty weights [17, 18] and a causal τ -curriculum [19]—directly address these failure modes.

Foundational methods.. We build on PINNs [8] and deep PDE solvers [9, 10]; on rough volatility [1, 5] and its multifactor Markovian lift [6, 7]; and on classical transform pricing [3, 20] for our independent, non-lifted forward-model ground truth.

3. Problem formulation

3.1. Rough Heston and the Markovian lift

Under rough Heston the spot price S and its instantaneous variance V satisfy

$$dS_t = \sqrt{V_t} S_t dW_t^S, \quad (1)$$

$$V_t = V_0 + \int_0^t K(t-s) \lambda(\theta - V_s) ds + \int_0^t K(t-s) \nu \sqrt{V_s} dW_s^V, \quad (2)$$

where W^S, W^V are Brownian motions with correlation ρ , and $K(t) = t^{\alpha-1}/\Gamma(\alpha)$ is the fractional kernel with $\alpha = H + \frac{1}{2}$ and $H < \frac{1}{2}$ [1, 5]. The variance is a Volterra process: non-Markovian, non-semimartingale, and admitting no finite-dimensional forward PDE. The parameter vector $(H, V_0, \theta, \lambda, \nu, \rho)$ is exactly what the calibration inverse problem must recover from market data; without a forward PDE there is no forward operator to embed inside that inverse problem at all.

The Markovian lift replaces K with $K(t) \approx \sum_{i=1}^n c_i e^{-x_i t}$, introducing n Ornstein–Uhlenbeck factors U^i with $dU_t^i = -x_i U_t^i dt + \sqrt{V_t} dW_t^V$ and $V_t = V_0 + \sum_i c_i U_t^i$ [6, 7]. The weights (c_i, x_i) are placed on a geometric quadrature grid [6]; the relative L^2 kernel error falls monotonically from 25.6% at $n=5$

to 0.39% at $n=100$, so raising n genuinely buys forward-model fidelity, at the cost of raising the forward-problem’s dimension by the same n .

3.2. The lifted forward PDE

The discounted European call price $P(t, x, u)$, where $x = \log S$ and $u = (u_1, \dots, u_n)$ collects the factor values, satisfies the $(n+1)$ -dimensional backward Kolmogorov PDE

$$\begin{aligned} 0 = & P_t + \left(r - \frac{1}{2}V\right) P_x + \frac{1}{2}V P_{xx} + \sum_{i=1}^n [-x_i u_i + \lambda(\theta - V)] P_{u_i} \\ & + \frac{1}{2}\nu^2 V \sum_{i,j=1}^n P_{u_i u_j} + \rho\nu V \sum_{i=1}^n P_{xu_i} - rP, \end{aligned} \quad (3)$$

with terminal condition $P(T, x, u) = \max(e^x - K, 0)$. Because all factors share the single Brownian motion W^V , the full double sum over (i, j) and the $(n+1)$ -dimensional cross term both appear; the PDE dimension is exactly $n+1$. At $n=4$ this is a five-dimensional forward problem; at $n=32$ it is thirty-three. Solving (3) for a single candidate parameter vector is one forward-model evaluation inside the outer calibration loop; a naive optimiser needs hundreds to thousands of these per fit (Section 6).

4. The PINN forward operator: architecture and training

4.1. Baseline network

We represent the forward map as a fully connected multilayer perceptron (MLP). Every hidden unit uses the tanh activation, which provides the smooth, high-order derivatives needed for automatic differentiation of the PDE residual; tanh is standard for PINN residual training and avoids the dead-neuron pathology of ReLU in this high-derivative setting [8]. All weight matrices are initialised with Xavier normal initialisation [21] and all biases with zeros.

The network maps $(\tau, x, u_1, \dots, u_n) \in \mathbb{R}^{2+n}$ to a scalar correction. For the single-point forward operator (parameters fixed) we use width $W=64$ and depth $D=4$ hidden layers; for the parametric forward operator used inside the inverse problem (Section 7.4) we increase to $W=96$, $D=5$. Table 1 summarises all architectural and training hyperparameters.

4.2. Hard-constrained initial-condition ansatz

We train in time-to-maturity $\tau = T - t$ so the terminal condition becomes an initial condition at $\tau=0$. Rather than penalise it softly, we enforce it exactly through the ansatz

$$P(\tau, x, u) = P_{\text{base}}(\tau, x) + \tau N_{\theta}(\tau, x, u), \quad (4)$$

where N_{θ} is the network output. The τ prefactor forces the learned correction to vanish at $\tau=0$ automatically, so the network never fights the initial condition. Without this, a soft penalty for the terminal payoff oscillates against the PDE residual and the forward operator fails to converge.

4.3. Integrated-variance baseline anchor

The baseline P_{base} is a Black–Scholes call price. A critical design choice is what volatility it is anchored at. Anchoring at the *spot* variance V_0 leaves a systematic level bias: the factors mean-revert from V_0 toward θ over the life of the option, so the model’s actual expected variance is higher than V_0 . We anchor instead at the model’s expected *integrated* variance

$$w(\tau) = \int_0^{\tau} \mathbb{E}[V_s] ds, \quad (5)$$

computed by a short pre-flight Monte Carlo (the same simulation used for factor collocation). The baseline then uses $\sigma(\tau) = \sqrt{w(\tau)/\tau}$, which correctly captures the mean-reversion effect; the network learns only the residual stochastic-volatility correction. This single change removed a uniform ~ 120 volbp level bias we observed with the V_0 anchor—an error that would otherwise propagate directly into every inverse-problem fit using this forward operator.

4.4. MC-informed factor collocation

The factor values u at a given maturity τ are not centred at zero: they mean-revert to a distribution set by the model dynamics. Sampling from a fixed symmetric box centred at zero causes the network to expend capacity on regions the dynamics never visit, and—critically—to under-cover the region governing the put-wing skew. Before training we run a short lifted Monte Carlo simulation (1,000 paths, 50 time steps) to estimate the mean and standard deviation of each factor at the target maturity; training collocation is then drawn from $\mathcal{N}(\hat{\mu}_i, \hat{\sigma}_i)$. An additional 20% of collocation points are

drawn at twice the estimated standard deviation to extend tail coverage. This change cut smile RMSE by roughly 45% (from ~ 210 to ~ 114 vol bp) relative to symmetric-box sampling.

4.5. Self-adaptive loss weighting

Three loss terms must be balanced at each training step: the PDE-residual MSE $\mathcal{L}_{\text{PDE}}$, and the MSEs at the deep-OTM ($P \rightarrow 0$) and deep-ITM ($P \rightarrow S - Ke^{-r\tau}$) boundaries. Fixed-weight balancing is brittle: the ITM boundary target is $O(1)$ while the OTM target is $O(0)$, so a single lumped weight over-emphasises the ITM term. We adopt the learnable homoscedastic-uncertainty scheme of Kendall et al. [17] in the self-adaptive spirit of McClenny and Braga-Neto [18]: each term $\mathcal{L}_k$ is assigned a learnable log-variance s_k and the objective is

$$\mathcal{L} = \sum_{k \in \{\text{PDE}, \text{OTM}, \text{ITM}\}} [e^{-s_k} \mathcal{L}_k + s_k]. \quad (6)$$

The s_k are optimised jointly with the network weights; large or noisy terms are down-weighted automatically. We never hand-tune a boundary weight.

4.6. Causal τ -curriculum

Forward operators that train simultaneously across all time-to-maturities can be pulled toward spurious solutions at long τ before the short- τ region has converged, because the residual at long τ depends causally on the short- τ solution [19]. We address this with a causal curriculum: training begins at $\tau_{\text{max}} = 0.15T$ and linearly ramps to the full T over the first half of training. The short- τ solution is therefore established before the long- τ residual can perturb it.

4.7. Training algorithm

Algorithm 1 summarises the full training procedure for the forward operator.

5. Evaluation methodology: trusting the forward operator

5.1. Two independent ground truths

An AI forward operator used inside an inverse problem is only as trustworthy as its own error against the true forward map. We hold the network to two independent references.

Algorithm 1 Training the PINN forward operator for the lifted rough-Heston PDE

Require: Model parameters $(V_0, \lambda, \theta, \nu, \rho)$; lift $(c_i, x_i)_{i=1}^n$; maturity T ; hyperparameters in Table 1

- 1: Initialise MLP N_θ (Xavier normal); learnable $s = [s_1, s_2, s_3] \leftarrow 0$
- 2: **Pre-flight MC:** simulate lifted SDE, record $(\hat{\mu}_i, \hat{\sigma}_i)$ for each factor and term structure $w(\tau)$ $\triangleright$ Sections 4.4, 4.3
- 3: **for** iteration $t = 1, \dots, N_{\text{iter}}$ **do**
- 4: $\tau_{\text{max}} \leftarrow \min(T, 0.15T + \frac{0.85T \cdot t}{0.5N_{\text{iter}}})$ $\triangleright$ τ -curriculum, Sec. 4.6
- 5: Sample interior: $\{(\tau_j, x_j, u_j)\}$ from $\text{Unif}([0, \tau_{\text{max}}]) \times \text{Unif}(x_{\text{lo}}, x_{\text{hi}}) \times \mathcal{N}(\hat{\mu}, \hat{\sigma})$
- 6: Sample boundary: $\{(\tau_j^{\text{bc}}, x_{\text{lo/hi}}, u_j^{\text{bc}})\}$
- 7: $\hat{P} \leftarrow P_{\text{base}}(\tau, x; w) + \tau N_\theta(\tau, x, u)$ $\triangleright$ Ansatz, Eq. (4)
- 8: $\mathcal{L}_{\text{PDE}} \leftarrow$ mean-squared PDE residual of $\hat{P}$ (Eq. (3))
- 9: $\mathcal{L}_{\text{OTM}} \leftarrow \|\hat{P}(\tau^{\text{bc}}, x_{\text{lo}}, u^{\text{bc}})\|^2$, $\mathcal{L}_{\text{ITM}} \leftarrow \|\hat{P}(\tau^{\text{bc}}, x_{\text{hi}}, u^{\text{bc}}) - (e^{x_{\text{hi}}} - Ke^{-r\tau^{\text{bc}}})\|^2$
- 10: $\mathcal{L} \leftarrow \sum_k [e^{-s_k} \mathcal{L}_k + s_k]$ $\triangleright$ Adaptive weighting, Eq. (6)
- 11: Backpropagate $\mathcal{L}$; clip gradients ($\ell_2 \leq 1.0$); Adam step on (N_θ, s) ; cosine LR anneal
- 12: **end for**

Fourier forward model. A semi-analytic pricer that solves the fractional Riccati equation for the rough Heston characteristic function [5] and inverts by Fourier integration [20]. It never uses the lift, so it is the ground truth of the *true, non-lifted model* ($n \rightarrow \infty$); in the flat-volatility limit it reproduces Black–Scholes to 3.7×10^{-7} in price.

Lifted Monte Carlo. An exponential-integrator Euler scheme that integrates the n -factor lifted SDE with the $-x_i U^i$ drift handled exactly:

$$U_{t+\Delta t}^i = e^{-x_i \Delta t} U_t^i + \int_t^{t+\Delta t} e^{-x_i(t+\Delta t-s)} \sqrt{V_s} dW_s^V. \quad (7)$$

The exponential-integrator is essential: the mean reversions (x_i) span roughly ten orders of magnitude for small H , and a naive Euler scheme diverges. The lifted MC matches Black–Scholes within Monte Carlo error (1,000 paths, 50 steps) and agrees with the Fourier forward model for large n . At finite n , the lifted MC is the *exact solution of the forward PDE the network is asked to solve*; the Fourier forward model is the exact solution of the true model.

5.2. Error decomposition

Rather than reporting a single “network-versus-true-model” figure, we decompose the error per strike and in price space (where the signed contributions add exactly),

$$\underbrace{(\text{PINN} - \text{Fourier})}_{\text{total forward error}} = \underbrace{(\text{PINN} - \text{MC})}_{\text{network solve-error}} + \underbrace{(\text{MC} - \text{Fourier})}_{\text{lift (model) error}}. \quad (8)$$

The *solve-error* measures how accurately the network solves the forward PDE it was given; it is the quantity training controls, and it is the AI method’s own contribution to inverse-problem error. The *lift-error* is a forward-model approximation that only increasing n can reduce; it is not the network’s responsibility, and no amount of network re-training removes it. Conflating the two gives a systematically pessimistic view of the AI method and leads to the wrong engineering response inside a calibration pipeline—e.g. training longer when the correct response is to raise n .

We report price RMSE in basis points of spot (px bp; $1 \text{ px bp} = 10^{-4} \times S$) where (8) is additive. Implied-vol equivalents (vol bp) are quoted where they aid interpretation. Note that the two components carry opposite signs, so their RMSE values do not add.

5.3. Homogeneity re-scaling

We train the network at fixed strike $K=1$ and read off the smile $C(1, K)$ through the call’s degree-one homogeneity:

$$C(1, K) = K C(1/K, 1), \quad (9)$$

which holds exactly because the variance dynamics are independent of the spot level. Dropping this re-scaling and comparing mismatched options inflates the apparent error to ~ 2700 vol bp; we verified this is a comparison artefact, not a modelling failure of the forward operator.

6. The calibration inverse problem: data and methods

6.1. Data

We use end-of-day BTC option quotes from Deribit, a cryptocurrency derivatives exchange and data provider, retrieved through its public API (no API key required). After filtering for positive bid–ask spreads and non-zero

open interest, the surface covers 13 maturities and 97 points spanning a moneyness range of roughly 0.6–1.7. We set $r=0$ (standard for crypto, since there is no clean carry) and infer the forward from the put–call parity of liquid near-the-money quotes. Implied volatilities are computed from mid prices with a robust Newton–Raphson Black–Scholes inverter. The inverse-problem objective throughout is the vega-weighted implied-volatility RMSE between market quotes and the forward operator’s output.

6.2. Three routes to the inverse problem

We compare three ways of solving the same calibration inverse problem.

1. **True-forward-model loop.** Differential evolution (60 generations, all 5 parameters) with the Fourier forward model inside the objective, followed by Nelder–Mead polish. This is our from-scratch, exact-forward-operator reference.
2. **AI-only.** Gradient descent on the model parameters using the parametric PINN as a differentiable forward operator, with zero calls to the true forward model.
3. **Hybrid.** The parametric PINN’s single-shot solution provides the initial guess; a short Nelder–Mead polish against the true forward model then sharpens it.

7. Results

7.1. The forward operator scales where the grid cannot

Our headline forward-operator experiment trains one architecture (width 64, depth 4) at $n=4, 8, 16, 32$ on the canonical BTC calibration at $T=0.15$, over three independent seeds (Table 2, Figure 1). The solve-error against the lifted MC grows only mildly: from 5.4 ± 1.0 to 10.7 ± 2.0 px bp—a doubling over an eight-fold rise in dimension. We resist calling it flat: an early single-seed run gave that impression, and three-seed statistics corrected us. With three seeds the error bars at adjacent n overlap, so the robust statement is the two-fold growth across the full range. Wall-clock cost stays bounded throughout (6–10 minutes on CPU, per-iteration time essentially flat at 55–62ms) because the factor-Hessian computation is vectorised over the full double sum.

Compare this against a finite-difference forward solver. A naive full-tensor grid on the same $(n+1)$ -dimensional PDE at a modest 50 nodes per

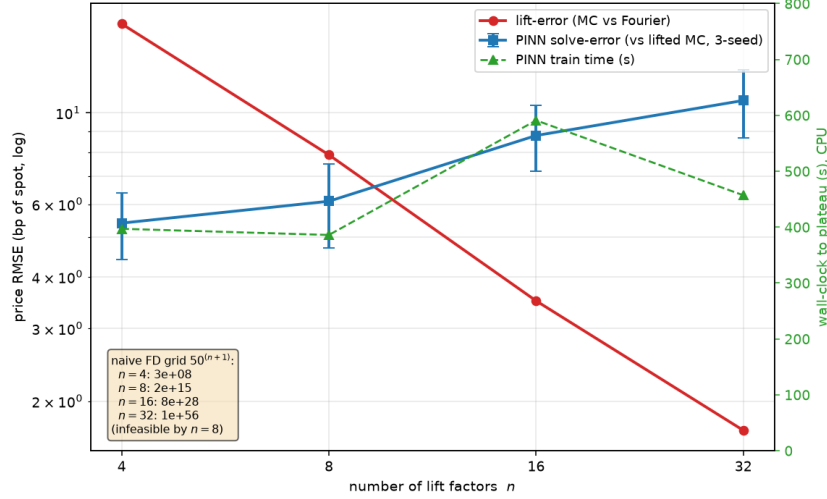


Figure 1: Dimension scaling of the forward operator. PINN solve-error (blue, 3-seed mean $\pm$ SD) grows sub-linearly and stays under ~ 11 px bp to $n=32$, while the lift-error (red) falls monotonically and training time (green) stays bounded. The inset shows the naive finite-difference grid size 50^{n+1} , which is infeasible by $n=8$.

dimension needs 50^{n+1} points: 3.1×10^8 at $n=4$, 2.0×10^{15} at $n=8$, and 1.2×10^{56} at $n=32$. Even $n=8$ —roughly 16 petabytes in double precision—is out of reach as a forward operator inside any iterative inverse-problem solver, while the network trains every n on a laptop. Structured schemes (sparse grids, ADI, low-rank tensor-train) reach further but still degrade with n ; none is standard or validated for this coupled ρ - ν PDE. The conservative verdict is that a competitive grid forward solver at $n=16$ – 32 is *impractical*, not provably impossible, as the engine of a calibration loop.

7.2. Error decomposition and a design rule for n

The decomposition (8) overturned how we read our own results. A single “network vs. true-model” implied-vol RMSE of 114–127 vol bp at $n=4$ *looks* like a failure of the AI forward operator. Split by (8), almost all of it is the lift: 37 vol bp of solve-error on top of 133 of lift-error (they partly cancel to a total of 121); with $\rho=0$ the solve-error is only 8 vol bp while the lift contributes the same 133. The network solves its forward PDE to well under 50 vol bp; what had appeared to be a network failure was the finite- n forward model catching up to reality.

Because the lift-error shrinks monotonically with n (133, 63, 27, 12 vol bp at $n=4, 8, 16, 32$) while the solve-error grows gently, the two curves cross. At $T \approx 0.15$ the crossover falls near $n=8-10$ (Figure 1), giving a concrete design rule for anyone embedding this forward operator in an inverse-problem solver: *below the crossover, more factors buy accuracy; above it, they only buy cost.* The crossover shifts with maturity: the lift-error grows with T (from 8.6 to 31.8 px bp at $n=4$ as T rises from 0.05 to 0.5), so longer-maturity calibration needs larger n .

Robustness sweeps over $T \in \{0.05, 0.15, 0.5\}$ and $H \in \{0.05, 0.10, 0.15\}$ confirm that the solve-error stays small and bounded everywhere (1–9.5 px bp across all configurations), while the lift-error varies predictably. The network does carry a mild directional bias at large n : at $n=32$ it systematically overprices ATM and call strikes by roughly +87 to +114 vol bp while the put wing is near zero; this bias grows with n and is a watch item for anyone deploying this forward operator at high n inside a calibration loop.

7.3. Solving the inverse problem: Bitcoin is genuinely rough

Calibrated to the full Deribit BTC surface with a full-budget optimiser (differential evolution, 60 generations, all 13 maturities, 97 points, followed by Nelder–Mead polish against the true Fourier forward model), rough Heston returns

$$H=0.090, \quad \rho=-0.792, \quad V_0=0.103, \quad \theta=0.253, \quad \lambda=2.567, \quad \nu=0.300,$$

at a vega-weighted implied-vol RMSE of 496 vol bp (Figure 2). The Hurst exponent is firmly rough and the correlation strongly negative—the signature of a crypto market with a steep, persistent left skew. We do not dress up the inverse-problem fit quality: at 496 vol bp it is coarse, and the per-maturity smiles in Figure 2(b) show where the model strains. That coarseness is a statement about the forward model’s expressiveness on wide, richly curved crypto smiles, not about the AI forward operator; it is orthogonal to the solve-error studied above.

7.4. A parametric AI forward operator inside the inverse problem

A single network cannot cover both the breadth of a parameter box and the accuracy of a point-trained forward operator. Table 3 and Figure 3 report the calibration experiment with three seeds.

The parametric network adds $(H, \nu, \rho, \lambda, \theta)$ as five additional inputs, holding V_0 and the maturity fixed; we use $n=10$ (the crossover rule, Section 7.2)

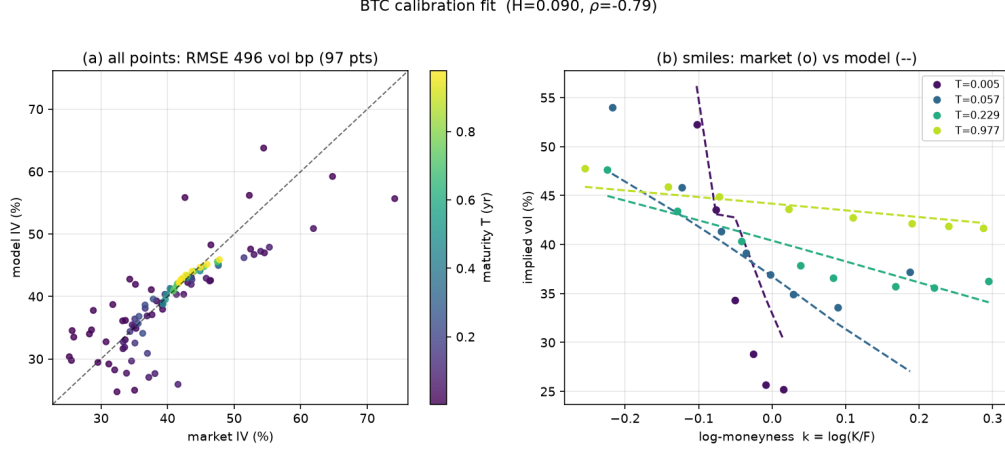


Figure 2: Rough Heston calibrated to the Deribit BTC surface (97 points, 13 maturities), solving the inverse problem with the true Fourier forward model. (a) Market versus model implied volatility at all points (dashed: identity; colour: maturity). (b) Per-maturity smiles for four representative maturities: market (markers) versus model (dashed). The residual spread reflects a model-expressiveness limitation on wide crypto smiles, not a forward-operator artefact.

and width 96, depth 5. At the box centre its solve-error is 11.0 ± 4.7 px bp versus 5.9 ± 1.2 for a single-point forward operator at the same n —roughly a two-fold penalty for generality. The penalty is uneven: it hovers near 9–11 px bp at typical parameter vectors but climbs to 47.7 in the rough low- H corner and 29.8 at high ν , where the PDE is stiffest—the generalisation limit of this AI forward operator, stated explicitly rather than averaged away.

Used as a standalone inverse-problem solver the parametric operator is fast (approximately 2s, zero true-model calls) but poor: 222 ± 20 volbp under the true model. Two factors explain this. First, the network fits its own imperfect forward map, not the market. Second, the single-slice BTC optimum prefers $\nu \approx 0.87$ and $\theta \approx 0.39$, which lie *outside* the training box, so the network cannot reach the true minimum—an identifiability issue of the training design, not of the underlying inverse problem. We would not recommend the standalone parametric operator as an inverse-problem solver.

Used as a warm start it is effective. Seeding a Nelder–Mead polish against the true forward model with the network’s parameter guess reaches 29.4 ± 7.1 volbp—on par with the from-scratch result on a 34–52% smile—in only 237 ± 46 true-forward-model evaluations and ≈ 23 s: roughly nine times fewer

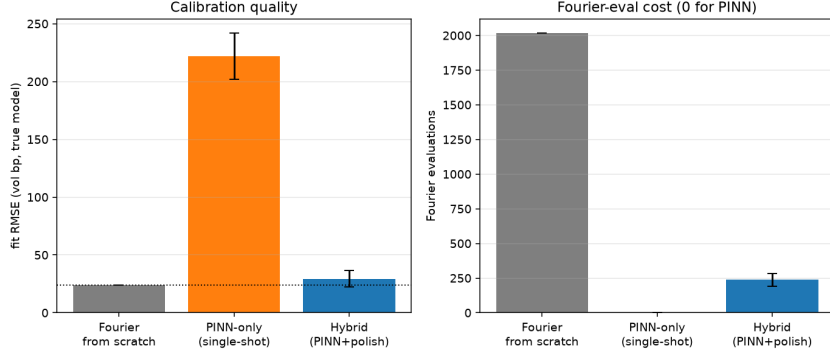


Figure 3: Hybrid solution of the inverse problem. Fit quality (left) and true-forward-model evaluation count (right) for the three methods. The hybrid matches from-scratch quality at $\sim 9\times$ fewer true-model evaluations; the AI-only route is fast but inaccurate.

expensive forward-model calls and seven times faster than solving the inverse problem from scratch. The value proposition is amortised: train the AI forward operator once, then solve each new instance of the inverse problem cheaply, with the exact forward model kept in the loop for a brief, decisive finish.

8. Discussion

Convergence of the AI forward operator.. The finding we care about most is that the network’s own error in solving the lifted forward PDE degrades only gently as the dimension grows, while grid methods become impractical as an inverse-problem engine. We state it in its honest, weak form: the solve-error is not constant in n —a claim an early single-seed run seemed to support and three-seed statistics refuted—it grows sub-linearly, roughly doubling from $n=4$ to $n=32$. And the grid comparison uses the naive 50^{n+1} count; sparse-grid, ADI, and tensor-train methods go further, so the defensible claim is *impractical, not impossible*. Even so, the contrast—sub-linear error and bounded wall-clock versus a grid that cannot be stored—is a concrete instance of the mesh-free advantage as a component of an inverse-problem pipeline, and we believe it is among the clearest such instances on an economically real, high-dimensional forward PDE.

Stability: four design choices are load-bearing.. We ablated each stabiliser explicitly. Without the hard-constrained ansatz, the soft IC penalty oscillates

against the residual and the forward operator fails to converge. Without the integrated-variance anchor, a uniform 120 vol bp level bias remains and propagates directly into every downstream calibration. Without MC-informed factor sampling, the smile flattens and RMSE roughly doubles. Without self-adaptive weighting or the τ -curriculum, seed variance is markedly higher (from 339 ± 62 to 310 ± 28 vol bp in an early stability comparison). Each of these choices is transferable to other AI forward operators for high-dimensional inverse problems.

A lesson about validating AI forward operators.. The error decomposition revealed something methodologically important. When a learned surrogate replaces an exact forward solver inside an inverse problem, it is dangerously easy to validate it against the wrong reference. Our network solves the finite- n lifted PDE, whose exact solution is the lifted Monte Carlo, not the $n \rightarrow \infty$ true model; comparing straight to the true model conflates network error with model error. We found the two carry opposite signs and that the historical “network-versus-true-model” gap was lift-dominated, not a failure of the AI method. *Validate an AI forward operator against a same-fidelity reference, report the two errors separately, and let their crossover set the approximation order:* none of this is special to rough volatility, and we urge it on anyone placing a learned forward operator inside an inverse problem built on an approximated governing equation.

Generalisation and identifiability inside the inverse problem.. A single network spread over five parameters is markedly less accurate than a point-trained one, worst in the rough, high-vol-of-vol corners where the PDE is stiffest—an explicit, quantified generalisation limit rather than an averaged headline number. Used alone as an inverse-problem solver it is additionally vulnerable to a training-box identifiability failure: if the true optimum lies outside the box the parametric operator was trained on, no amount of gradient descent inside that box recovers it. This is not a limitation of the inverse problem itself, which remains well posed once regularised by the Fourier forward model; it is a limitation of trusting the AI forward operator unsupervised. The warm-start hybrid sidesteps exactly this failure mode by keeping the true forward model as the final arbiter.

Limitations and watch items.. Our evidence rests on a single market snapshot (one Deribit BTC surface), and the parametric and calibration studies fix the maturity to a single slice. Timings are single-machine CPU figures at

moderate budgets. The mild directional solve-bias at large n (ATM and call over-pricing growing to ~ 100 vol bp at $n=32$) is a watch item for high- n deployments inside a calibration loop. The sub-linear trend need not continue arbitrarily far.

Future work.. Natural extensions include: (i) adding the maturity as a network input so a single forward operator serves the whole surface’s inverse problem at once; (ii) scaling width, depth, and GPU training to shrink the parametric corner penalty and widen the training box, directly addressing the identifiability failure mode above; (iii) extending to other rough models (rough Bergomi) and to American and path-dependent payoffs, where the inverse problem is harder still; (iv) investigating whether residual reweighting or a correction term resolves the high- n directional bias; (v) a formal stability/identifiability analysis of the calibration inverse problem itself as $n \rightarrow \infty$, complementing the purely empirical treatment given here.

9. Conclusion

We have characterised a physics-informed neural network as a forward operator for the rough-volatility calibration inverse problem, on the $(n+1)$ -dimensional Markovian-lifted PDE that makes the forward problem well-posed in the first place. The network’s solve-error grows only sub-linearly in the lifted dimension—staying below eleven price basis points to $n=32$ on the BTC calibration—while naive finite-difference forward solvers become infeasible by $n=8$, and we identify the four training choices that make this convergence behaviour hold. Separating solve-error from lift-error is essential to correctly attribute inverse-problem error and yields a concrete, maturity-dependent rule for the factor count. Applied to live Bitcoin options the recovered forward model is genuinely rough ($H \approx 0.09$), and a parametric extension of the forward operator, while not accurate enough to solve the inverse problem independently, is an effective warm start that solves it at roughly nine times fewer expensive forward-model evaluations. The methodological message is that an AI forward operator earns its place inside an inverse-problem pipeline only once its own error is separated from the model error it is approximating—and that, done this way, it degrades gracefully where grid methods cannot follow.

CRedit authorship contribution statement

Aviral Sharma: Conceptualisation, Methodology, Software, Validation, Formal analysis, Writing – original draft, Writing – review & editing. **Kamal Preet Singh:** Writing – review & editing, Funding acquisition. **Vipul Sharma:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Data availability statement

The BTC option data were retrieved from the Deribit public REST API (no API key required) at <https://www.deribit.com/api/v2>. The raw data snapshot used in the experiments, the calibrated parameter file, and the complete PyTorch implementation are available at [\[repository URL upon acceptance\]](#).

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this manuscript the authors used Claude (Anthropic) to assist with prose drafting and structural organisation. After using this tool, the authors reviewed and edited all content as needed and take full responsibility for the accuracy and integrity of the published work. The mathematical derivations, experimental results, code, and all scientific claims are the authors' own.

References

- [1] J. Gatheral, T. Jaisson, M. Rosenbaum, Volatility is rough, *Quantitative Finance* 18 (6) (2018) 933–949. doi:10.1080/14697688.2017.1393551.
- [2] M. Bennedsen, A. Lunde, M. S. Pakkanen, Decoupling the short- and long-term behavior of stochastic volatility, *Journal of Financial Econometrics* 20 (5) (2022) 961–1006. doi:10.1093/jjfinec/nbaa049.
- [3] S. L. Heston, A closed-form solution for options with stochastic volatility with applications to bond and currency options, *Review of Financial Studies* 6 (2) (1993) 327–343. doi:10.1093/rfs/6.2.327.
- [4] F. Comte, E. Renault, Long memory in continuous-time stochastic volatility models, *Mathematical Finance* 8 (4) (1998) 291–323. doi:10.1111/1467-9965.00057.
- [5] O. El Euch, M. Rosenbaum, The characteristic function of rough Heston models, *Mathematical Finance* 29 (1) (2019) 3–38. doi:10.1111/mafi.12173.
- [6] E. Abi Jaber, O. El Euch, Multifactor approximation of rough volatility models, *SIAM Journal on Financial Mathematics* 10 (2) (2019) 309–349. doi:10.1137/18M1170236.
- [7] E. Abi Jaber, Lifting the Heston model, *Quantitative Finance* 19 (12) (2019) 1995–2013. doi:10.1080/14697688.2019.1615113.
- [8] M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *Journal of Computational Physics* 378 (2019) 686–707. doi:10.1016/j.jcp.2018.10.045.
- [9] J. Sirignano, K. Spiliopoulos, DGM: A deep learning algorithm for solving partial differential equations, *Journal of Computational Physics* 375 (2018) 1339–1364. doi:10.1016/j.jcp.2018.08.029.
- [10] J. Han, A. Jentzen, W. E, Solving high-dimensional partial differential equations using deep learning, *Proceedings of the National Academy of Sciences* 115 (34) (2018) 8505–8510. doi:10.1073/pnas.1718942115.

- [11] A. Papapantoleon, J. Rou, A time-stepping deep gradient flow method for option pricing in (rough) diffusion models, *Quantitative Finance* 25 (12) (2025) 2009–2020. doi:10.1080/14697688.2025.2572318.
- [12] A. Jacquier, v. Zurič, Random neural networks for rough volatility, preprint, arXiv:2305.01035 (2023). arXiv:2305.01035.
- [13] C. Bayer, B. Horvath, A. Muguruza, B. Stemper, M. Tomas, On deep calibration of (rough) stochastic volatility models, preprint, arXiv:1908.08806 (2019). arXiv:1908.08806.
- [14] B. Horvath, A. Muguruza, M. Tomas, Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models, *Quantitative Finance* 21 (1) (2021) 11–27. doi:10.1080/14697688.2020.1817974.
- [15] A. S. Krishnapriyan, A. Gholami, S. Zhe, R. M. Kirby, M. W. Mahoney, Characterizing possible failure modes in physics-informed neural networks, *Advances in Neural Information Processing Systems (NeurIPS)* 34 (2021) 26548–26560.
- [16] S. Wang, X. Yu, P. Perdikaris, When and why PINNs fail to train: a neural tangent kernel perspective, *Journal of Computational Physics* 449 (2022) 110768. doi:10.1016/j.jcp.2021.110768.
- [17] A. Kendall, Y. Gal, R. Cipolla, Multi-task learning using uncertainty to weigh losses for scene geometry and semantics, *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2018) 7482–7491 doi:10.1109/CVPR.2018.00781.
- [18] L. D. McClenny, U. M. Braga-Neto, Self-adaptive physics-informed neural networks, *Journal of Computational Physics* 474 (2023) 111722. doi:10.1016/j.jcp.2022.111722.
- [19] S. Wang, S. Sankaran, P. Perdikaris, Respecting causality for training physics-informed neural networks, *Computer Methods in Applied Mechanics and Engineering* 421 (2024) 116813. doi:10.1016/j.cma.2024.116813.

- [20] P. Carr, D. B. Madan, Option valuation using the fast Fourier transform, *Journal of Computational Finance* 2 (4) (1999) 61–73. doi:10.21314/JCF.1999.043.
- [21] X. Glorot, Y. Bengio, Understanding the difficulty of training deep feed-forward neural networks, in: *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010, pp. 249–256.
- [22] D. P. Kingma, J. Ba, Adam: a method for stochastic optimization, *International Conference on Learning Representations (ICLR)* ArXiv:1412.6980 (2015).

Table 1: Architecture and training hyperparameters of the PINN forward operator. Values in parentheses are for the parametric extension (Section 7.4).

Component	Value
<i>Architecture</i>	
Network type	Fully connected MLP
Hidden layers	4 (5 for parametric)
Neurons per layer	64 (96 for parametric)
Activation	tanh (all hidden layers)
Weight initialisation	Xavier normal
Bias initialisation	Zero
Input dimension	$2 + n$ for single-point; $2 + n + 5$ for parametric
Output dimension	1 (price correction N_θ)
Floating-point precision	float64 (PyTorch)
<i>Optimisation</i>	
Optimiser	Adam [22]
Initial learning rate	10^{-3}
LR schedule	Cosine annealing to 0 over all iterations
Gradient clipping	ℓ_2 norm ≤ 1.0
Training iterations	10,000–20,000 (single-point); 20,000–30,000 (parametric)
<i>Collocation sampling (per iteration)</i>	
Interior collocation	2,000 uniform + 500 near-strike (total 2,500)
Boundary points	400 (split OTM / ITM)
Log-moneyness domain	$[\log K - 1.2, \log K + 1.2]$
Factor sampling	MC-empirical mean / std (Section 4.4)
Tail coverage fraction	20% of u -points drawn at $2 \times$ std
<i>Loss weighting</i>	
Adaptive scheme	Learnable log-variance (homoscedastic uncertainty)
Terms balanced	PDE residual MSE, OTM boundary MSE, ITM boundary MSE
<i>τ-curriculum</i>	
Initial horizon	$\tau_0 = 0.15 T$
Ramp fraction	Linear growth to T over first 50% of iterations
<i>Hardware</i>	
Platform	CPU (single machine); code is GPU-compatible (<code>torch.device</code>)

Table 2: Dimension scaling of the forward operator: single-point PINN at $T=0.15$ on the canonical BTC calibration. Solve-error is 3-seed mean $\pm$ SD in price basis points of spot; lift-error and FD grid size are deterministic.

n	solve-error (px bp)	lift-error (px bp)	train time (s)	FD grid $50^{(n+1)}$
4	5.4 ± 1.0	16.4	397	3.1×10^8
8	6.1 ± 1.4	7.9	386	2.0×10^{15}
16	8.8 ± 1.6	3.5	591	7.6×10^{28}
32	10.7 ± 2.0	1.7	457	1.2×10^{56}

Table 3: Solving the calibration inverse problem for the BTC $T \approx 0.15$ slice, three methods, 3 PINN seeds. All fit-quality figures are judged by the true Fourier forward model over the same feasible region.

Method	Fit RMSE (vol bp)	Wall-clock (s)	True-model c
True-forward-model loop, from scratch	23.7	168	2018
AI-only (single-shot)	222 ± 20	~ 2	0
Hybrid (AI warm start + true-model polish)	29.4 ± 7.1	23	237 ± 46